COMPUTER SECURITY

PROJECT: AI/ML-ENHANCED MULTI-FACTOR AUTHENTICATION TO MITIGATE PASSWORD GUESSING ATTACKS

Total Marks: 20
Turnitin Allowed: 5%
Deadline: 29th May 2026@5PM
Submission: eLeap
Maximum Report Length: 20 pages

Use of Generative AI

If students utilize any generative AI tools (e.g., ChatGPT or similar systems), they must include a separate section at the end of the report clearly describing how the AI tools were used and the extent of their usage in completing the assignment. This section should explain which parts of the work involved AI assistance and how the outputs were reviewed, modified, or integrated by the students.

1. Project Overview

Password guessing attacks such as brute-force attacks, dictionary attacks, and AI-assisted password cracking remain major threats to modern authentication systems. Traditional password-based authentication mechanisms are vulnerable because attackers can repeatedly attempt different passwords until the correct one is found.
To address this challenge, this project requires students to design and implement an AI/ML-enhanced Multi-Factor Authentication (MFA) system capable of detecting suspicious login attempts and mitigating password guessing attacks.
The project emphasizes system design, algorithm development, implementation, and evaluation of the proposed security solution.

Project Questions and Marks Allocation

Question 1: Problem Statement (2 Marks)

Students must formulate a clear problem statement based on password guessing attacks discussed in the assignment.
The problem statement should explain:

- what password guessing attacks are
- why password-only authentication systems are vulnerable
- limitations of existing authentication mechanisms
- why an AI/ML-enhanced MFA system can help mitigate these attacks

Students may briefly refer to insights from the assignment, but no detailed literature review is required.

Question 2: Proposed AI/ML-Enhanced MFA Solution and Algorithm Design (8 Marks)

Students must design an AI/ML-enhanced Multi-Factor Authentication (MFA) system to mitigate password guessing attacks.
The proposed solution may be inspired by or adapted from the research articles used in the assignment, but students do not need to re-explain the articles or repeat the literature review.
Students must focus on designing and explaining their security system.

1. System Architecture and Design (4 Marks)
Students must present the architecture of the proposed authentication system.
The design must include the following components:

- Password authentication layer
- Multi-Factor Authentication mechanism (e.g., OTP, authenticator app)
- AI/ML-based suspicious login detection module
- Mitigation response mechanism

Students must include:

- a system architecture diagram
- explanation of each system component
- description of how the system mitigates password guessing attacks

Example system flow:
User enters username and password
↓
System verifies password
↓
AI/ML engine evaluates login behavior
↓
Low risk → Standard MFA
High risk → Strong MFA / delay / block

2. Algorithm or AI/ML Technique (4 Marks)

Students must clearly explain the algorithm or technique used to detect suspicious login behavior.
The explanation must include:

- description of the algorithm or ML model
- input features used in detection (e.g., login attempts, timing, IP patterns)
- step-by-step explanation of how the algorithm works
- pseudocode or flowchart of the algorithm

Examples of acceptable techniques:

- Decision Tree
- Random Forest
- Logistic Regression
- Rule-based risk scoring
- Behavioral anomaly detection

Students must clearly demonstrate how the algorithm detects password guessing attempts and triggers mitigation mechanisms such as MFA or login blocking.

Question 3: System Implementation (4 Marks)

Students must implement a working prototype demonstrating the proposed solution.
The system must include:

- login interface
- password authentication
- multi-factor authentication verification
- AI/ML-based suspicious login detection
- mitigation response mechanism

Examples of mitigation responses include:

- triggering MFA verification
- delaying login attempts
- temporarily locking accounts
- blocking suspicious login attempts

Students must include:

- screenshots of the system
- explanation of code implementation
- description of system components

Suggested technologies:

- Python
- Flask / Streamlit / Tkinter
- Scikit-learn
- PyOTP
- SQLite / JSON database

Question 4: Testing, Correctness and Effectiveness Analysis (3 Marks)
Students must demonstrate whether their system mitigates password guessing attacks.
They must test their system using different scenarios such as:

- normal login attempt
- incorrect password attempt
- repeated login attempts
- simulated password guessing attack

Students must provide:

- testing tables
- system response results
- explanation of whether mitigation successfully prevents password guessing attacks

Example testing table:

Scenario	**System Response**	**Result**
Normal login	Access granted	Success
Wrong password	Retry allowed	Safe
Multiple attempts	MFA triggered	Attack mitigated
Automated guessing	Login blocked	Attack prevented

Students must clearly explain why the proposed solution can be considered correct and effective.

Question 5: Recommendations and Future Improvements 3 Mark)
Students must propose improvements for their system.
Examples may include:

- improving ML detection accuracy
- reducing false positives
- integrating behavioral biometrics
- integrating passwordless authentication
- combining with blockchain-based identity systems

RUBRIC

Criteria	Marks	100-90%	90-70%	70-40%	Below 40%
Problem Statement	/2	Problem is clearly defined with strong explanation of password guessing threats, weaknesses of password-only authentication, and motivation for AI/ML-enhanced MFA. Logical and technically sound.	Problem generally clear but explanation of threats or motivation lacks depth.	Problem partially explained with weak connection to password guessing attacks.	Problem unclear, incorrect, or missing key elements.
Proposed Solution Design & Algorithm	/8	Solution design is clearly derived from research articles studied in the assignment. Students correctly adapt concepts from the literature to design an AI/ML-enhanced MFA system. System architecture, workflow, and mitigation strategy are clearly explained. Algorithm or model is fully presented using pseudocode, diagrams, or flowcharts. Proper citations are included showing how the solution was inspired by the articles.	Solution is related to literature but explanation of how the articles influenced the design is limited. Algorithm explanation partially complete.	Solution loosely related to articles with minimal adaptation. Algorithm explanation weak or incomplete.	Solution not connected to research articles or algorithm not explained.
System Implementation	/4	Functional prototype demonstrating password authentication, MFA, and AI/ML-based risk detection. Implementation clearly explained with screenshots, system interface, and code description. Mitigation mechanisms (e.g., MFA triggering, blocking suspicious attempts) clearly demonstrated.	Prototype mostly functional but some components incomplete or insufficiently explained.	Partial implementation with limited system functionality.	Implementation missing or non-functional.
Testing and Correctness Analysis	/3	System tested using multiple scenarios. Tables and figures clearly demonstrate system behavior under password guessing attempts. Explanation clearly shows why the solution mitigates password guessing attacks.	Some testing provided but analysis limited.	Minimal testing with weak explanation of system effectiveness.	No meaningful testing or correctness analysis.
Recommendations and Improvements	/3	Practical and well-reasoned suggestions for improving the system or extending the solution proposed in the literature.	Basic recommendations provided but lacking depth.	Very limited recommendations.	No recommendations provided.

Sample Project implementation
AI/ML-Enhanced Multi-Factor Authentication to Mitigate Password Guessing Attacks
It includes:

- login with username/password
- simple **ML/risk-based suspicious login detection**
- OTP-based MFA
- mitigation response:
    - normal MFA
    - stronger check for suspicious attempts
    - temporary blocking after repeated failed attempts

This is a **teaching sample**, not a production-ready security system.

1. Project Idea
The system works like this:

1. User enters username and password
2. System checks login behavior features
3. A simple ML/risk model decides whether the login is:
    - low risk
    - medium risk
    - high risk
4. If risk is high, the system can:
    - force MFA
    - delay login
    - temporarily block the user
5. If OTP is correct, access is granted

2. Example Features Used for Detection
Students can use these input features:

Feature	Meaning
failed_attempts	number of failed attempts for that user
short_interval	whether attempts are happening too quickly
unknown_device	whether device is unknown
unusual_hour	whether login is at unusual time
password_match	whether password is correct

The model or rule engine uses these to estimate suspicious behavior.

3. Simple System Workflow
User enters username and password
↓
System checks password
↓
System extracts login behavior features
↓
Risk engine predicts low / medium / high risk
↓
Low risk → OTP MFA
Medium risk → OTP MFA + delay
High risk → temporary block
↓
If OTP correct → access granted

Else → access denied

4. Suggested Folder Structure

```
project/
│
├── app.py
├── users.json
├── otp_store.json
├── templates/
│   ├── login.html
│   ├── otp.html
│   ├── blocked.html
│   └── success.html
└── static/
    └── style.css
```

5. Sample users.json

```
{
 "student1": {
   "password": "Secure123!",
   "email": "student1@example.com",
   "known_device": "laptop-01"
 },
 "student2": {
   "password": "StrongPass456!",
   "email": "student2@example.com",
   "known_device": "phone-02"
 }
}
```

6. Sample Flask Implementation

app.py

```
from flask import Flask, render_template, request, redirect, url_for, session
import json
import os
import random
import time
from datetime import datetime

app = Flask(__name__)
app.secret_key = "sample_secret_key_for_demo_only"

USERS_FILE = "users.json"
OTP_FILE = "otp_store.json"

# Track user attempts in memory for demo
login_tracker = {}

BLOCK_THRESHOLD = 5
BLOCK_DURATION = 60  # seconds
```

```
def load_users():
    if os.path.exists(USERS_FILE):
        with open(USERS_FILE, "r") as f:
            return json.load(f)
    return {}


def load_otp_store():
    if os.path.exists(OTP_FILE):
        with open(OTP_FILE, "r") as f:
            return json.load(f)
    return {}


def save_otp_store(data):
    with open(OTP_FILE, "w") as f:
        json.dump(data, f, indent=4)


def generate_otp():
    return str(random.randint(100000, 999999))


def get_current_hour():
    return datetime.now().hour


def is_unusual_hour():
    # Example rule: unusual if login between 12 AM and 5 AM
    hour = get_current_hour()
    return 1 if hour < 5 else 0


def initialize_user_tracker(username):
    if username not in login_tracker:
        login_tracker[username] = {
            "failed_attempts": 0,
            "last_attempt_time": 0,
            "blocked_until": 0
        }


def extract_features(username, device_id, password_correct):
    initialize_user_tracker(username)

    tracker = login_tracker[username]
    current_time = time.time()

    short_interval = 1 if current_time - tracker["last_attempt_time"] < 5 else 0
    unusual_hour = is_unusual_hour()
```

```
    users = load_users()
    unknown_device = 1

    if username in users:
        if device_id == users[username]["known_device"]:
            unknown_device = 0

    features = {
        "failed_attempts": tracker["failed_attempts"],
        "short_interval": short_interval,
        "unknown_device": unknown_device,
        "unusual_hour": unusual_hour,
        "password_match": 1 if password_correct else 0
    }

    tracker["last_attempt_time"] = current_time
    return features


def simple_risk_engine(features):
    """
    Simple rule-based AI/risk scoring engine for teaching purposes.
    Students can replace this with Decision Tree / Random Forest / Logistic Regression.
    """
    score = 0

    if features["failed_attempts"] >= 3:
        score += 2
    if features["short_interval"] == 1:
        score += 2
    if features["unknown_device"] == 1:
        score += 1
    if features["unusual_hour"] == 1:
        score += 1
    if features["password_match"] == 0:
        score += 2

    if score >= 5:
        return "high", score
    elif score >= 3:
        return "medium", score
    else:
        return "low", score


@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]
```

```
device_id = request.form["device_id"]

users = load_users()
initialize_user_tracker(username)
tracker = login_tracker[username]

# Check if blocked
if time.time() < tracker["blocked_until"]:
  remaining = int(tracker["blocked_until"] - time.time())
  return render_template("blocked.html", remaining=remaining)

password_correct = False
if username in users and password == users[username]["password"]:
  password_correct = True

features = extract_features(username, device_id, password_correct)
risk_level, risk_score = simple_risk_engine(features)

# If wrong password, increment failed attempts
if not password_correct:
  tracker["failed_attempts"] += 1

  # High-risk repeated guessing -> block
  if tracker["failed_attempts"] >= BLOCK_THRESHOLD:
    tracker["blocked_until"] = time.time() + BLOCK_DURATION
    return render_template("blocked.html", remaining=BLOCK_DURATION)

  return render_template(
    "login.html",
    error=f"Invalid password. Risk level: {risk_level}, score: {risk_score}"
  )

# Correct password entered
# Reset failed attempts if correct
tracker["failed_attempts"] = 0

# Mitigation logic
if risk_level == "high":
  tracker["blocked_until"] = time.time() + BLOCK_DURATION
  return render_template("blocked.html", remaining=BLOCK_DURATION)

elif risk_level == "medium":
  # Delay suspicious login
  time.sleep(3)

# Generate OTP
otp = generate_otp()
otp_store = load_otp_store()
otp_store[username] = otp
save_otp_store(otp_store)
```

```
        session["username"] = username
        session["risk_level"] = risk_level

        # For demo only: show OTP on screen/log
        print(f"[DEMO OTP for {username}]: {otp}")

        return redirect(url_for("otp_verification"))

    return render_template("login.html")


@app.route("/otp", methods=["GET", "POST"])
def otp_verification():
    if "username" not in session:
        return redirect(url_for("login"))

    username = session["username"]
    risk_level = session.get("risk_level", "low")

    if request.method == "POST":
        entered_otp = request.form["otp"]
        otp_store = load_otp_store()
        correct_otp = otp_store.get(username)

        if entered_otp == correct_otp:
            otp_store.pop(username, None)
            save_otp_store(otp_store)
            return render_template("success.html", username=username, risk_level=risk_level)

        return render_template("otp.html", error="Invalid OTP", risk_level=risk_level)

    return render_template("otp.html", risk_level=risk_level)


@app.route("/success")
def success():
    return render_template("success.html")


if __name__ == "__main__":
    app.run(debug=True)
```

7. HTML Templates
templates/login.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Secure Login</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
```

```
  <div class="box">
    <h2>AI/ML-Enhanced MFA Login</h2>

    {% if error %}
      <p class="error">{{ error }}</p>
    {% endif %}

    <form method="POST">
      <label>Username</label>
      <input type="text" name="username" required>

      <label>Password</label>
      <input type="password" name="password" required>

      <label>Device ID</label>
      <input type="text" name="device_id" placeholder="e.g. laptop-01" required>

      <button type="submit">Login</button>
    </form>
  </div>
</body>
</html>
```

templates/otp.html

```
<!DOCTYPE html>
<html>
<head>
  <title>OTP Verification</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <div class="box">
    <h2>OTP Verification</h2>
    <p>Risk level: <strong>{{ risk_level }}</strong></p>

    {% if error %}
      <p class="error">{{ error }}</p>
    {% endif %}

    <form method="POST">
      <label>Enter OTP</label>
      <input type="text" name="otp" required>
      <button type="submit">Verify OTP</button>
    </form>
  </div>
</body>
</html>
```

templates/blocked.html

```
<!DOCTYPE html>
<html>
```

```
<head>
  <title>Blocked</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <div class="box">
    <h2>Access Temporarily Blocked</h2>
    <p>Your login was identified as suspicious.</p>
    <p>Please try again after <strong>{{ remaining }}</strong> seconds.</p>
  </div>
</body>
</html>
```

templates/success.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Login Success</title>
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
  <div class="box">
    <h2>Login Successful</h2>
    <p>Welcome, <strong>{{ username }}</strong>.</p>
    <p>Authenticated with MFA.</p>
    <p>Risk level during login: <strong>{{ risk_level }}</strong></p>
  </div>
</body>
</html>
```

8. Sample CSS

static/style.css

```
body {
  font-family: Arial, sans-serif;
  background: #f3f6fb;
  margin: 0;
  padding: 0;
}

.box {
  width: 420px;
  margin: 80px auto;
  background: white;
  padding: 25px;
  border-radius: 10px;
  box-shadow: 0 2px 10px rgba(0,0,0,0.1);
}

h2 {
  text-align: center;
  margin-bottom: 20px;
```

```
}

label {
  display: block;
  margin-top: 12px;
  margin-bottom: 5px;
}

input {
  width: 100%;
  padding: 10px;
  box-sizing: border-box;
}

button {
  width: 100%;
  padding: 12px;
  margin-top: 18px;
  border: none;
  background: #2563eb;
  color: white;
  font-size: 15px;
  border-radius: 6px;
  cursor: pointer;
}

button:hover {
  background: #1d4ed8;
}

.error {
  color: red;
  font-weight: bold;
}
```

9. How This Sample Mitigates Password Guessing

This sample includes several mitigation mechanisms:

A. Password + MFA

Even if password is guessed, attacker still needs OTP.

B. Suspicious Login Detection

The system looks at:

- failed attempts
- rapid repeated attempts
- unknown device
- unusual login hour

C. Delay for Medium Risk

Suspicious behavior gets slowed down.

D. Temporary Blocking for High Risk

Repeated guessing attempts cause blocking.

10. Sample Pseudocode for the Report

Students can include something like this in Question 2.
Input: username, password, device_id
Output: allow, request_otp, delay, or block

1. Receive login request
2. Check whether user is currently blocked
3. Verify password correctness
4. Extract login behavior features:
   - failed attempts
   - short interval
   - unknown device
   - unusual hour
   - password match
5. Send features to risk engine
6. Compute risk level:
   - low
   - medium
   - high
7. If password incorrect:
   increment failed attempts
   if threshold exceeded:
      block user
   else:
      deny login
8. If password correct:
   if risk = high:
      block login
   if risk = medium:
      delay request
   generate OTP
   verify OTP
9. If OTP correct:
   grant access
  else:
   deny access

11. Example Testing Scenarios
Students should test the system with a table like this:

Scenario	Username	Password	Device	Expected Result
Normal login	student1	correct	known	OTP requested, login success
Wrong password	student1	wrong	known	Invalid password
Rapid guessing	student1	wrong many times	known	Blocked
Correct password, unknown device	student1	correct	unknown	Medium/high risk, OTP forced
Correct password at unusual hour	student1	correct	known	Risk increased, extra checking

12. Example Correctness Analysis

Students can write something like this:

The solution is considered effective because password-only authentication is not used alone. Even when the correct password is entered, the system still requires OTP verification. In addition, suspicious login behavior such as repeated failed attempts, short intervals between attempts, unknown devices, and unusual login time increases the risk score. This allows the system to delay or block likely password guessing attempts before account compromise occurs.

13. How Students Can Improve This Sample

Students can extend this implementation by:

- replacing the rule-based risk engine with a real ML model
- training a Decision Tree or Random Forest classifier
- adding CAPTCHA after repeated attempts
- storing hashed passwords instead of plain text
- using real email OTP delivery
- tracking IP addresses
- adding behavioral biometrics

14. Important Note for Students

This sample is only a guide. Students should:

- modify the design
- justify their chosen features
- connect the design to the articles they reviewed in the assignment
- explain why their implementation mitigates password guessing

They should not submit this unchanged.

If you want, I can next give you a sample ML-based version using Decision Tree with a small login-attempt dataset, which would be more advanced and closer to your project title.

End of the Project Description